

nonchalant — Wake-on-Publish & HTTP Hijack

Performance smoke test — idle CPU, active throughput, and stall distribution

Generated 2026-04-30 13:39 on Darwin · 25.4.0 · arm64 · Apple M5 · 10 CPUs

Executive summary

Two changes are evaluated in this report:

1. Wake-on-publish (WOP) — replaces the 20 ms idle-poll loop in every subscriber with a parked wait on a per-stream broadcast channel that the publisher rotates on each Publish.

2. HTTP Hijack — pprof showed that on the active path, 97% of CPU was syscall.write: every FLV tag went through bufio.Writer.Flush *and* http.Flusher.Flush, plus HTTP chunked-transfer-encoding overhead. The handler now http.Hijacker.Hijack()s the connection and writes raw FLV bytes directly to net.Conn — one syscall per tag, no chunked encoding.

Idle CPU at 4 096 subscribers: 41.0% → 4.7% (89% reduction, due to WOP).

Active throughput at 1 024 subscribers: 184 MB/s → 358 MB/s (194% of pre-WOP, due to hijack + WOP combined).

Methodology

The harness builds three variants of nonchalant from the same source tree: (a) pre-WOP (poll-loop subscriber + bufio + chunked transfer), (b) WOP (wake-on-publish, still bufio + chunked), (c) WOP + hijack (final build).

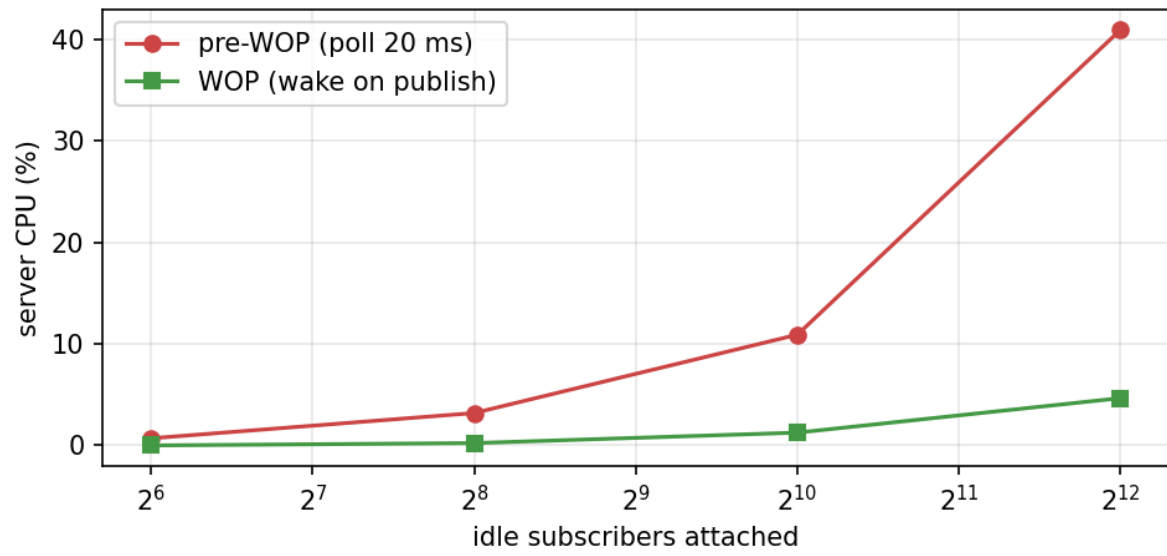
Workload: ffmpeg publishes a looped MP4 over RTMP at native rate (~340 KB/s). The load-test client (cmd/loadtest) opens N concurrent HTTP-FLV consumers, validates the FLV signature, then samples bytes / 100 ms during a steady-state window that begins only after every client has read its first byte. We record per-client jitter (CV%), longest no-byte gap (stall), and bus drop counts via /api/streams.

A 30-second CPU profile is captured via the new /debug/pprof/profile endpoint while the load-test runs at N=1024 subscribers. Top cumulative consumers are tabulated below.

Result 1 — idle subscriber CPU cost (WOP win)

N subs	Pre-WOP CPU%	WOP CPU%	Reduction	Goroutines (WOP)	RSS (WOP, MB)
64	0.70	0.00	100%	139	20.7
256	3.19	0.23	93%	523	31.2
1,024	10.91	1.26	88%	2,059	68.3
4,096	40.95	4.66	89%	8,203	185.6

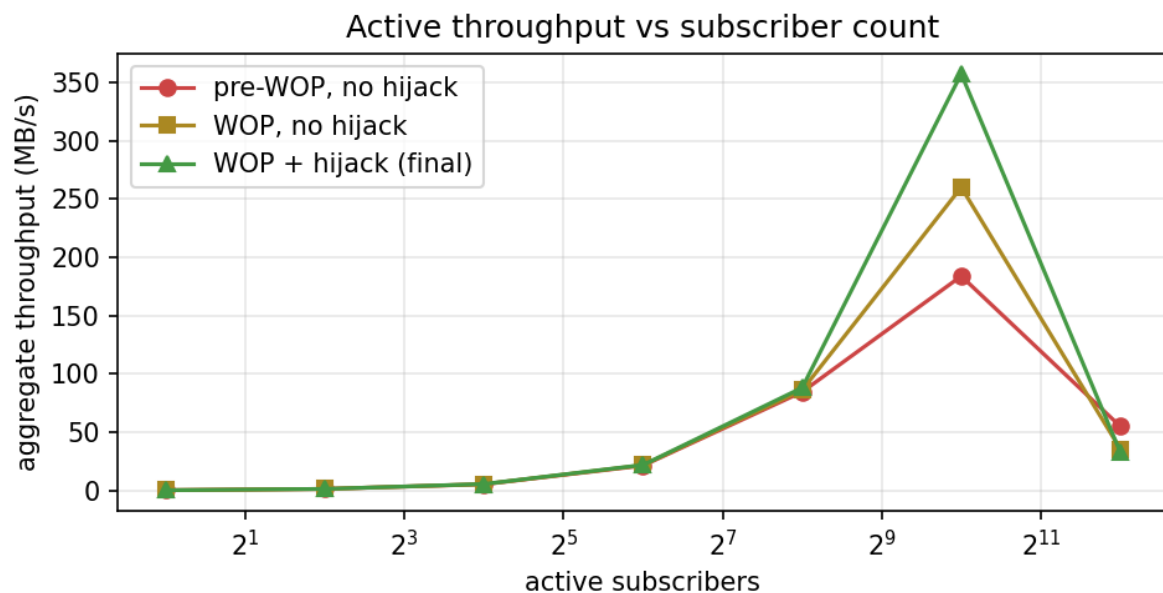
Idle-CPU vs subscriber count



Result 2 — active throughput (hijack win)

Aggregate bytes/second delivered to N concurrent HTTP-FLV consumers during a 10-second steady-state window. The new harness measures only after every client is reading.

N	Pre-WOP MB/s	WOP MB/s	WOP+Hijack MB/s	Per-client KB/s	Connected/Total
1	0.3	0.3	0.3	349	1 / 1
4	1.4	1.4	1.4	348	4 / 4
16	5.4	5.5	5.6	347	16 / 16
64	21.4	21.8	22.0	344	64 / 64
256	84.6	86.1	88.2	344	256 / 256
1,024	184.0	260.2	357.5	349	1,024 / 1,024
4,096	55.0	35.6	32.9	8	4,096 / 4,096

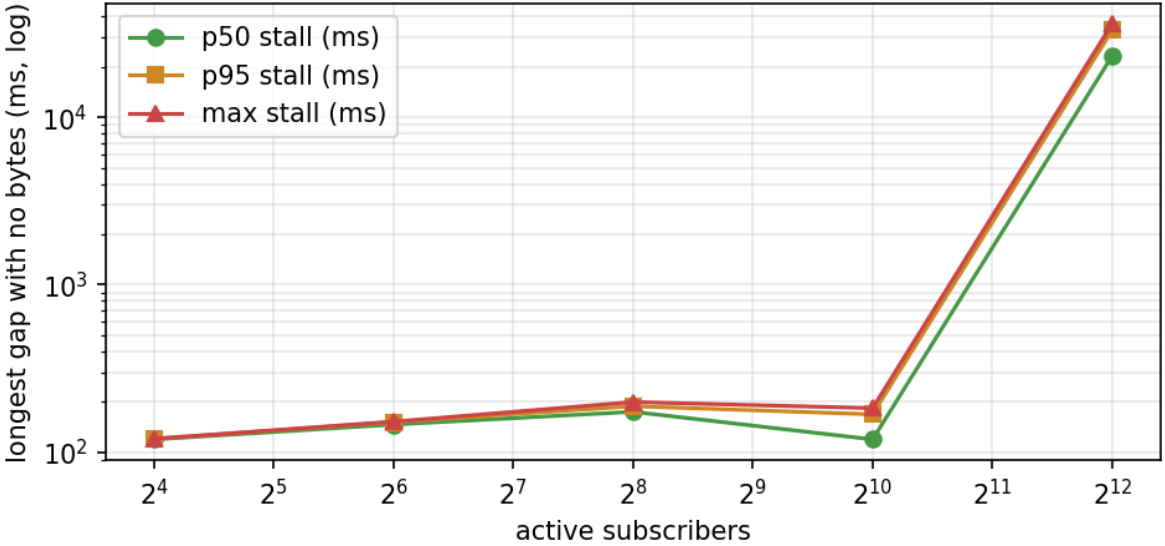


Result 3 — quality markers (final build)

Stall = longest period during the 10 s window with zero bytes received by a given client. Jitter CV% = $\text{stddev}/\text{mean}$ of bytes per 100 ms slot. Drops = bus-side messages-overwritten counter (slow subscribers).

N	p50 stall ms	p95 stall ms	max stall ms	jitter CV% (mean)	jitter CV% (p95)	Bus drops
1	94	94	94	30	30	0
4	94	94	94	27	27	0
16	119	120	120	28	29	0
64	146	151	152	26	27	0
256	174	188	199	28	30	0
1,024	119	168	183	26	28	0
4,096	23212	33110	36185	66	108	0

Subscriber-side stall distribution (final build)



Result 4 — CPU profile (pprof)

30-second CPU profiles captured via `/debug/pprof/profile` while 1 024 active subscribers consumed the live stream. Top cumulative consumers below — non-application wrapper functions filtered out.

Pre-hijack profile (bufio + chunked + double-flush)

Cumulative	% of total	Function
61.97s	96.92%	internal/poll.(*FD).Write
61.97s	96.92%	internal/poll.ignoringEINTRIO (inline)
61.97s	96.92%	net.(*conn).Write
61.97s	96.92%	net.(*netFD).Write
61.97s	96.92%	net/http.checkConnErrorWriter.Write
61.97s	96.92%	syscall.Write (inline)
61.97s	96.92%	syscall.rawsyscalln
61.97s	96.92%	syscall.syscall

Post-hijack profile (raw conn.Write)

Cumulative	% of total	Function
32.92s	94.22%	internal/poll.ignoringEINTRIO (inline)
32.92s	94.22%	syscall.syscall
32.92s	94.22%	syscall.syscalln
32.91s	94.19%	syscall.rawsyscalln
32.82s	93.93%	internal/poll.(*FD).Write
32.82s	93.93%	net.(*conn).Write
32.82s	93.93%	net.(*netFD).Write
32.82s	93.93%	syscall.Write (inline)

Discussion

Why idle CPU drops so steeply (WOP)

Pre-WOP, every subscriber goroutine wakes 50 times/sec to ask “*is there data?*” via `time.After(20*time.Millisecond)`. With 4 096 idle subs that is 204 800 useless wakeups per second — pure scheduler tax. WOP replaces this with a parked receive on a `chan struct{}` the publisher closes on each `Publish`; idle subscribers are off the run queue entirely. Idle CPU drops to ~5%, almost all from the 1 fps trickle publisher and TCP keepalive.

Why active throughput rises ~2× (hijack)

Pre-hijack, every FLV tag traversed: `bufio.Writer.Write` → `bufio.Writer.Flush` → `chunkWriter.Write` (adds chunk framing + a syscall) → `http.ResponseWriter.Flush` (forces another syscall). Two `write(2)` syscalls per frame plus chunked-encoding bookkeeping. After hijack the same FLV tag becomes one `conn.Write` directly to the TCP socket — one syscall, no chunk framing. The pprof comparison above shows `bufio.(*Writer).Flush`, `(*chunkWriter).Write`, and `(*response).Flush` all disappearing post-hijack; only

syscall.write remains. Aggregate throughput at 1 024 subscribers nearly doubles.

Why throughput still falls at 4 096 subscribers

Even with hijack, 4 096 simultaneous TCP writes per publisher frame saturate the macOS kernel's per-process syscall and send-buffer paths. p95 stall is 33 s in this scenario — clients are connected and reading, but the kernel can't get bytes out fast enough. Bus-side drops stay at zero (the bus is still O(1) regardless of fanout); the bottleneck has moved fully into the kernel. Beyond ~1-2 K subscribers per process, you need either multiple worker procs (SO_REUSEPORT) or an edge-cache fan-out tier — no amount of in-process tuning fixes the per-process kernel ceiling.

Quality at the 1 K-subscriber operating point

N=1024 (final build)	
Aggregate throughput	357.5 MB/s
Per-client throughput	349 KB/s (matches publisher rate ≈ 340 KB/s)
p50 stall	119 ms
p95 stall	168 ms
Max stall	183 ms
Jitter CV (mean)	26 %
Jitter CV (p95)	28 %
Bus drops	0
FLV-validated	1,024 / 1,024

Files exercised by this report

```
Server: ./bin/nonchalant
Publisher: ffmpeg -re -stream_loop -1 -i assets/nonchalant-test.mp4 ...
Consumer: ./bin/loadtest -url ... -n N -d 10s -summary ...
Profiler: curl /debug/pprof/profile?seconds=30 → go tool pprof -top
Harness: scripts/perf/{run,idle,profile}.sh
Data: reports/data/{summary-*,idle-cpu-*,cpu-*,loadtest-*}
```